GRADUATE ENGINEER TRAINEE – 2026

Programming and DSA Practice Paper

Total Questions: 13	**Sections:** 3	**Languages:** Java or Python

Instructions

- Attempt all questions.
- Sections 1 and 3 may be answered in Java or Python.
- Section 2 questions are based on Java. Write the exact output of each program.
- Write complete, compilable code. Assume input is valid unless the question states otherwise.

SECTION 1: DSA PROGRAMMING QUESTIONS (Questions 1–4)

Algorithmic problem solving. A node class or input array is given where needed. Write only the logic that solves the problem; building a data structure from scratch is not required.

Question 1: Reverse a Singly Linked List

Given the head of a singly linked list, reverse the list and return the head of the reversed list. The reversal must be done by changing the `next` links of the existing nodes. Do not create new nodes or copy values into a new list.

Node definition:

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}
```

Constraints: 0 ≤ number of nodes ≤ 100000. Expected time complexity: O(n). Expected auxiliary space: O(1).

Sample Input	Sample Output
`1 -> 2 -> 3 -> 4 -> 5 -> NULL`	`5 -> 4 -> 3 -> 2 -> 1 -> NULL`
`10 -> NULL`	`10 -> NULL`

Question 2: Detect a Cycle in a Linked List

Given the head of a singly linked list, determine whether the list contains a cycle. A cycle exists if some node can be reached again by continuously following the `next` pointer. Return `true` if a cycle exists, otherwise return `false`. The list must not be modified.

Node definition:

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}
```

Constraints: 0 ≤ number of nodes ≤ 100000. Expected time complexity: O(n). Expected auxiliary space: O(1).

Sample Input	Sample Output
`3 -> 2 -> 0 -> -4` `(the next of node -4 points to node 2)`	`true`
`1 -> 2 -> 3 -> NULL`	`false`

Question 3: Find the Middle Element of a Linked List

Given the head of a non-empty singly linked list, return the value stored in the middle node. If the list has an even number of nodes, there are two middle nodes; return the value of the second one.

Node definition:

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}
```

Constraints: 1 ≤ number of nodes ≤ 100000. Expected time complexity: O(n). Expected auxiliary space: O(1).

Sample Input	Sample Output
`1 -> 2 -> 3 -> 4 -> 5 -> NULL`	`3`
`1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL`	`4`

Question 4: Maximum Profit from a Single Stock Transaction

An array `prices` is given, where `prices[i]` is the price of a stock on day `i`. You may buy the stock on one day and sell it on a **later** day, completing at most one transaction. Find the maximum profit that can be earned. If no profit is possible, the answer is 0.

Constraints: 1 ≤ prices.length ≤ 100000; 0 ≤ prices[i] ≤ 10000. Expected time complexity: O(n).

Sample Input	Sample Output
`prices = [7, 1, 5, 3, 6, 4]`	`5`
`prices = [7, 6, 4, 3, 1]`	`0`

SECTION 2: JAVA OUTPUT-BASED QUESTIONS (Questions 5–9)

Each program compiles and runs without errors (Java 8 or later). Write the exact output, line by line, in the order it is printed.

Question 5

What is the exact output of the following program?

```
class Vehicle {
    static String type() {
        return "Vehicle";
    }

    String name() {
        return "Generic Vehicle";
    }

    void show() {
        System.out.println(type() + " | " + name());
    }
}

class Car extends Vehicle {
    static String type() {
        return "Car";
    }

    @Override
    String name() {
        return "Sedan";
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        Car c = (Car) v;

        System.out.println(v.type());
        System.out.println(v.name());
        System.out.println(c.type());
        v.show();
        c.show();
    }
}
```

Question 6

What is the exact output of the following program?

```
public class Main {
    static void show(Object o) {
        System.out.println("Object");
    }

    static void show(String s) {
        System.out.println("String");
    }

    static void show(int n) {
        System.out.println("int");
    }

    static void show(long n) {
        System.out.println("long");
    }

    public static void main(String[] args) {
        Object o = null;
        Integer i = 7;

        show(null);
        show((Object) "Java");
        show('A');
        show(10L);
        show(5);
        show(o);
        show(i);
    }
}
```

Question 7

What is the exact output of the following program?

```
class Base {
    static {
        System.out.println("Base static block");
    }

    {
        System.out.println("Base instance block");
    }

    Base() {
        System.out.println("Base constructor");
        init();
    }

    void init() {
        System.out.println("Base init");
    }
}

class Derived extends Base {
    static {
        System.out.println("Derived static block");
    }

    private int value = 50;

    {
        System.out.println("Derived instance block, value = " + value);
    }

    Derived() {
        super();
        System.out.println("Derived constructor, value = " + value);
    }

    @Override
    void init() {
        System.out.println("Derived init, value = " + value);
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("main started");
        new Derived();
        System.out.println("---");
        new Derived();
    }
}
```

Question 8

What is the exact output of the following program?

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<String> words = Arrays.asList(
            "banana", "Apple", "cherry", "apple",
            "Banana", "date", "cherry", "fig");

        List<String> result = words.stream()
            .map(String::toLowerCase)
            .filter(w -> w.length() > 3)
            .distinct()
            .sorted(Comparator.comparing(String::length)
                              .thenComparing(Comparator.reverseOrder()))
            .collect(Collectors.toList());

        long count = words.stream()
            .filter(w -> w.length() > 3)
            .map(String::toLowerCase)
            .distinct()
            .count();

        System.out.println(result);
        System.out.println(count);
    }
}
```

Question 9

What is the exact output of the following program?

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(3, 8, 5, 12, 7, 2);

        int sum = nums.stream()
            .filter(n -> n % 2 == 1)
            .map(n -> n * n)
            .reduce(0, Integer::sum);

        Optional<Integer> max = nums.stream()
            .filter(n -> n > 10)
            .reduce(Integer::max);

        Optional<Integer> none = nums.stream()
            .filter(n -> n > 20)
            .reduce(Integer::max);

        int product = nums.stream()
            .limit(3)
            .reduce(1, (a, b) -> a * b);

        System.out.println(sum);
        System.out.println(max.get());
        System.out.println(none.isPresent());
        System.out.println(product);

        Stream<Integer> s = nums.stream()
            .peek(n -> System.out.println("Visited " + n))
            .filter(n -> n > 6);

        System.out.println("Pipeline created");
        System.out.println(s.findFirst().get());
    }
}
```

SECTION 3: DATA STRUCTURE IMPLEMENTATION (Questions 10–13)

Implementation questions. Build each data structure from scratch in Java or Python, including the class definitions, and write a short driver program that calls every method. Method names may follow the convention of the language you use (for example, insert_at_beginning in Python).

Question 10: Doubly Linked List

Implement a doubly linked list of integers. Do not use `LinkedList`, `ArrayList`, `deque` or any other built-in list class. Each node must hold links to both its previous and next nodes. Provide the following methods:

- `insertAtBeginning(value)` – insert a new node at the head.
- `insertAtEnd(value)` – insert a new node at the tail.
- `insertAtPosition(position, value)` – insert at a 1-based position. Valid positions are 1 to size + 1. Report an error for any other position.
- `delete(value)` – delete the first node that holds the given value. Report if the value is not found.
- `search(value)` – return the 1-based position of the first match, or -1 if not found.
- `displayForward()` – print the values from head to tail.
- `displayBackward()` – print the values from tail to head.

Requirements: Handle the empty list (for example, deleting from or displaying an empty list must not fail) and the single-node case (deleting the only node must leave a valid empty list). Keep the head and tail references correct after every operation.

Question 11: Stack Implementation Using an Array

Implement a stack of integers using a fixed-size array. The capacity is given when the stack is created. Do not use `Stack`, `Deque`, `ArrayList` or any other built-in stack or list class. Provide the following methods:

- `push(value)` – add a value on top of the stack.
- `pop()` – remove and return the top value.
- `peek()` – return the top value without removing it.
- `isEmpty()` – return true if the stack has no elements.
- `isFull()` – return true if the stack has reached its capacity.
- `display()` – print all elements from top to bottom.

Requirements: Handle stack overflow (push on a full stack) and stack underflow (pop or peek on an empty stack) by reporting a clear message or raising an appropriate exception, without corrupting the stack.

Question 12: Binary Tree Implementation

Implement a binary tree of integers using linked nodes, where each node has a value, a left child and a right child. The `insert(value)` method must place the new node at the first vacant position found in level order (top to bottom, left to right), so that the tree is filled level by level. Provide the following methods:

- `insert(value)` – insert a value as described above.
- `search(value)` – return true if the value exists in the tree, otherwise false.
- `preorderTraversal()` – print values in Root, Left, Right order.
- `inorderTraversal()` – print values in Left, Root, Right order.
- `postorderTraversal()` – print values in Left, Right, Root order.
- `levelOrderTraversal()` – print values level by level, from left to right.

Requirements: The tree nodes and the tree class must be your own implementation. A built-in queue may be used only as the helper for `insert` and `levelOrderTraversal`. All methods must work correctly on an empty tree.

Question 13: Graph Implementation Using an Adjacency List

Implement an undirected, unweighted graph whose vertices are integers, using an adjacency list. Built-in list or map classes may be used to store the adjacency lists, but no graph library may be used. Provide the following methods:

- `addVertex(v)` – add a vertex. Ignore it if the vertex already exists.
- `addEdge(u, v)` – add an undirected edge between two existing vertices. Report an error if either vertex does not exist. Do not add a duplicate edge.
- `removeEdge(u, v)` – remove the edge between two vertices from both adjacency lists. Report if the edge does not exist.
- `BFS(start)` – print vertices in breadth-first order starting from the given vertex.
- `DFS(start)` – print vertices in depth-first order starting from the given vertex.
- `displayGraph()` – print every vertex followed by its list of neighbours.

Requirements: The graph may be disconnected. `BFS` and `DFS` from a start vertex must visit only the vertices reachable from it, and the program must also provide a way to traverse every component of a disconnected graph. Vertices with no edges must still appear in `displayGraph`.